



شُراحُ الشِّعر

- Shura7 Al-sh3er -

Standard RAG Vs Graph RAG

Abstract

This project, **Shora7 Al-sh3er**, is an expert system designed to help users understand and interpret the Muallaqat al-Ashr using two retrieval approaches: Standard RAG and Graph RAG. The system was developed to address the difficulty many readers face when trying to understand Arabic poetry and its linguistic meanings. To build the dataset, OCR tools were used to extract text from Muallaqat books, followed by text cleaning and verse-based chunking. In the Standard RAG pipeline, semantic embeddings and vector retrieval are used to find relevant verses. In the Graph RAG pipeline, relationships between verses, meanings, and grammatical concepts are represented using a graph structure for deeper contextual understanding. A large language model is then used to generate accurate and context-aware explanations. The system provides an accessible way to explore Arabic poetry by bridging modern readers with the linguistic richness of the Muallaqat.

By Sarah Alowjan, Haya Alwizrah

Content:

Introduction: 3

Data Sets:..... 3

Methodology: 4

 Embedding: 4

 Standard RAG: 4

 Graph RAG: 5

 System Workflow: 6

Evaluation:..... 7

 Automated Metrics: 7

 LLM as a Judge:..... 7

Conclusion 8

Introduction:

Today, many readers find classical Arabic poetry difficult to understand because of its complex vocabulary and linguistic style, especially the Muallaqat al-Ashr.

For this reason, we developed **Shora7 Al-sh3er**, an expert system for analyzing and interpreting the Muallaqat using both Standard RAG (vector-based) and Graph RAG (relationship-based).

The system helps users understand Arabic poetry by retrieving explanations, meanings, and grammatical relations from the Muallaqat dataset.

Data Sets:

For the dataset construction, we used two OCR technologies: Mistral AI and Qara to extract text from books of the Muallaqat Al-Ashr. We compared the outputs of both models and found that Mistral AI produced faster and more accurate results. Therefore, we relied on it to process two PDF books containing the Muallaqat Al-Ashr.

After the OCR stage, the extracted text contained noisy elements such as headers, page numbers, broken formatting, and duplicated lines. To improve the quality of the dataset, we manually cleaned and normalized the text before preparing it for retrieval.

Finally, the poems were segmented using verse-based chunking, where each poetic verse was treated as an independent chunk. This chunking strategy helped preserve the semantic meaning and poetic structure, improving retrieval accuracy for both the Standard RAG and Graph RAG architectures.

Currently, the system focuses on only 3 poetries:

- Al-Asha
- Antara ibn Shaddad
- Labid ibn Rabiah

Methodology:

Embedding:

To represent the semantic meaning of the poetic verses, we used the Arabic embedding model “Sarah0001/Arabic_embed_model” The model is a distilled version of the “Omartificial-Intelligence-Space/Harrier-Arabic-Matryoshka-270m” to make it more lightweight and efficient for retrieval tasks.

The embedding model transforms each verse into a vector representation that reflects the meaning of the text instead of depending only on exact keywords. This allows the system to retrieve verses and explanations based on contextual similarity, which is especially important when dealing with Arabic poetry and complex linguistic expressions.

After generating the embeddings, all vectors were stored in the vector database to support semantic search within the Standard RAG pipeline. This process enabled the system to retrieve the most relevant poetic verses and contextual information in response to user queries.

Standard RAG:

The Standard Retrieval-Augmented Generation (RAG) architecture was implemented to enable semantic retrieval and contextual answer generation for the Muallaqat dataset.

After converting the poetic verses into embeddings, the vectors were stored in ChromaDB, which was used as the vector database for semantic similarity search. ChromaDB allows the system to efficiently retrieve the most relevant verse chunks based on the meaning of the user’s query rather than exact keyword matching.

When a user submits a question, the query is first transformed into an embedding using the same embedding model. The system then performs similarity search inside ChromaDB to retrieve the closest and most contextually relevant poetic verses.

The retrieved context is then passed to the language model OpenAI “gpt-4o-mini”, which generates the final response. The model analyzes the retrieved content and produces explanations, interpretations, and answers in natural language while remaining grounded in the retrieved content.

This architecture improves answer accuracy and reduces hallucination by ensuring that the language model relies on relevant retrieved information from the dataset instead of depending only on its internal knowledge.

Graph RAG:

In addition to the standard vector-based retrieval pipeline, we implemented Graph RAG architecture to capture the structural and semantic relationships within Arabic poetry.

The graph system was built using Neo4j as a knowledge graph database. Unlike traditional RAG systems that rely mainly on semantic similarity, Graph RAG focuses on modeling relationships between different poetic components.

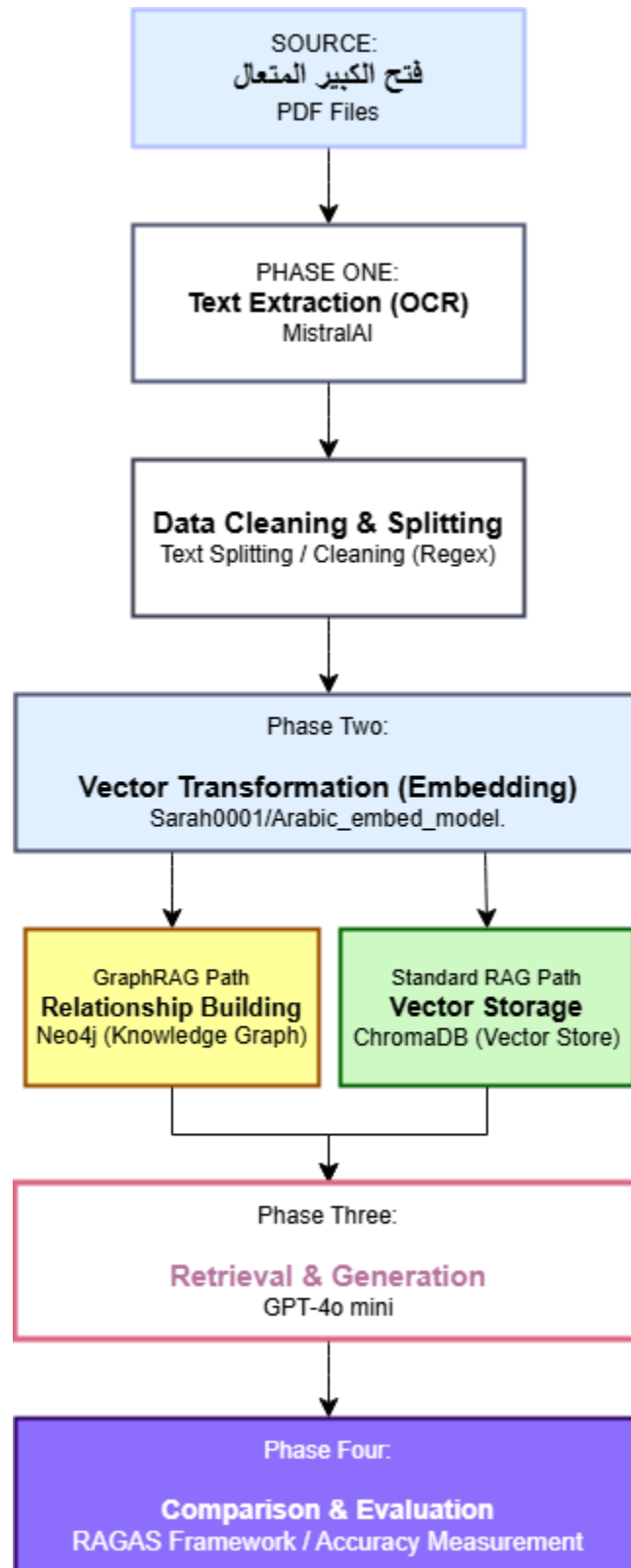
To prepare the data for the graph structure, we used the large language model “gpt-4o-mini” to analyze and structure the poetic verses before inserting them into the graph database. The model was used to divide and organize the information into meaningful components such as verses, grammatical analysis, and semantic meanings.

Each element was represented as nodes and relationships inside the graph, enabling the system to understand how linguistic and semantic concepts are connected throughout the poems.

When a user submits a query, the system traverses the graph structure to retrieve connected and contextually related information rather than retrieving isolated text chunks only. This allows the system to provide deeper contextual understanding and more coherent interpretations of the Muallaqat.

Finally, the retrieved graph context was passed again to the same language model, “gpt-4o-mini”, to generate natural language explanations and answers grounded in the graph-based relationships and retrieved information.

System Workflow:



Evaluation:

To assess the performance of our RAG system, we implemented a structured evaluation pipeline based on both automated metrics and LLM-based judgment. The evaluation process was used to measure the quality of both retrieval and answer generation in the system.

Automated Metrics:

We used the RAGAS evaluation framework, which provides standardized metrics for analyzing Retrieval-Augmented Generation systems. The evaluation dataset was first constructed from an Excel file containing questions and corresponding ground-truth answers. For each question, the system generates an answer using the RAG pipeline along with the retrieved context, which is then used for evaluation.

We used the following metrics:

- **Faithfulness:** This metric measures whether the generated answer is factually consistent with the retrieved context. It checks if the model avoids hallucination and only uses information present in the provided context.
- **Answer Relevancy:** This evaluates how relevant the generated answer is to the user's question. It ensures that the response directly addresses the query without deviating into unrelated information.
- **Context Precision:** This measures the quality of the retrieved context by checking how much of the retrieved information is useful for answering the question. A high precision score means that most retrieved chunks are useful for answering the question.
- **Context Recall:** evaluates whether the retrieval system successfully captures all necessary information required to answer the question. It measures how completely the retrieved context is with respect to the ground truth.

	Standard RAG	Graph RAG
Faithfulness	0.2750	0.3110
Answer Relevancy	0.5270	0.5410
Context Precision	1.0000	1.0000
Context Recall	0.6230	0.3300

Graph RAG achieved slightly better faithfulness and answer relevancy scores because the graph structure preserves relationships between verses and meanings. On the other hand, Standard RAG achieved higher context recall due to the broader retrieval behavior of vector similarity search.

LLM as a Judge:

In addition to RAGAS metrics, we implemented a custom LLM judge function that evaluates each response based on three criteria:

- Accuracy of the answer
- Degree of adherence to the provided context
- Relevance to the original question

The model assigns a score from 1 to 100 and returns the evaluation in a structured JSON format. This provides an additional qualitative layer of assessment beyond automated metrics.

	Standard RAG	Graph RAG
Accuracy	75	80
Degree of adherence	64	70
Relevance	76	77

By combining RAGAS metrics with LLM-based evaluation, the system provides a comprehensive assessment of both retrieval quality and answer generation performance, ensuring reliable evaluation for both Standard RAG and Graph RAG architectures.

Conclusion

This project, Shora7 Al-sh3er, demonstrates an effective approach to understanding and analyzing the Muallaqat al-Ashr by combining Standard RAG and Graph RAG architectures. By integrating OCR-processed datasets, semantic embeddings, and structured knowledge representation, the system successfully combines modern AI techniques with classical Arabic poetry analysis.

The Standard RAG component provides strong semantic retrieval based on vector similarity, while Graph RAG enhances structural understanding by modeling relationships between verses, grammar, and meanings. Together, both approaches enable more accurate, context-aware, and meaningful interpretations.

Overall, the system shows how combining retrieval-based AI with large language models can improve access to complex literary texts and make classical Arabic poetry more understandable and accessible to modern users.